

Entradas digitais



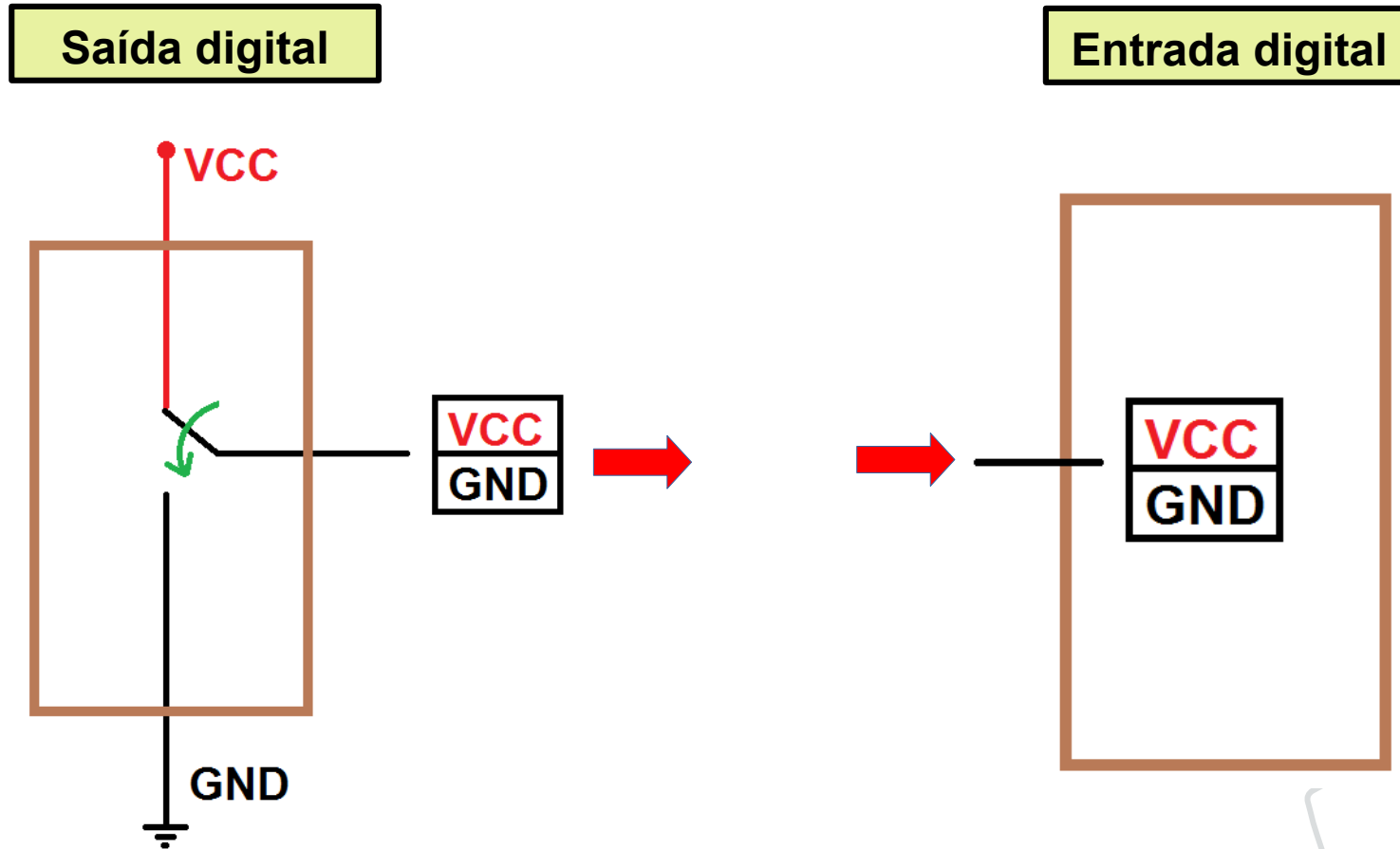
Entrada digitais



Chave

- Um circuito ligado a uma entrada digital deve possuir somente dois estados (0 ou 1).
- A tensão deve ser compatível com a entrada.

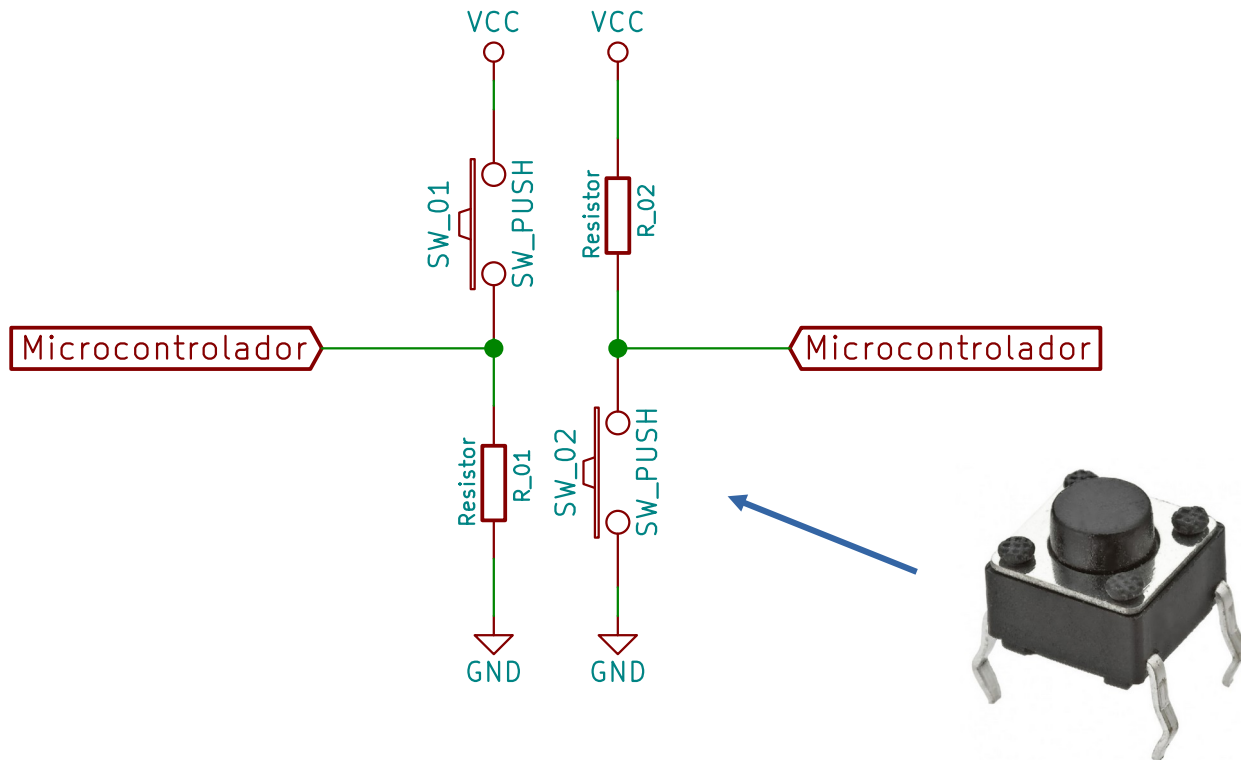
Entradas e saídas digitais



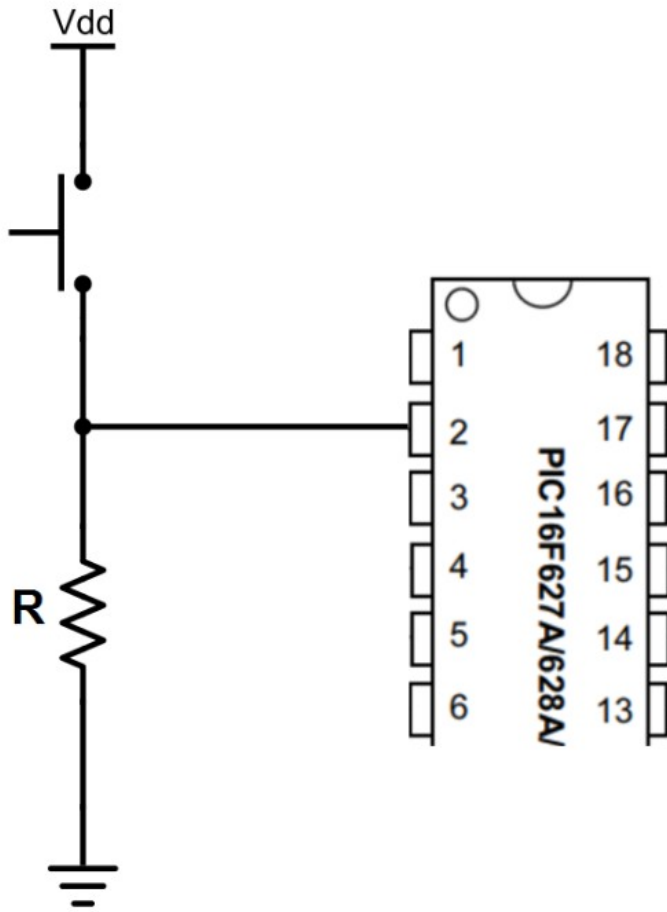
Os níveis de tensão devem ser compatíveis.

Entrada digitais

- O circuito mais simples utiliza botão ou chave.
 - A ligação pode ser do tipo Pull-up ou Pull-down.



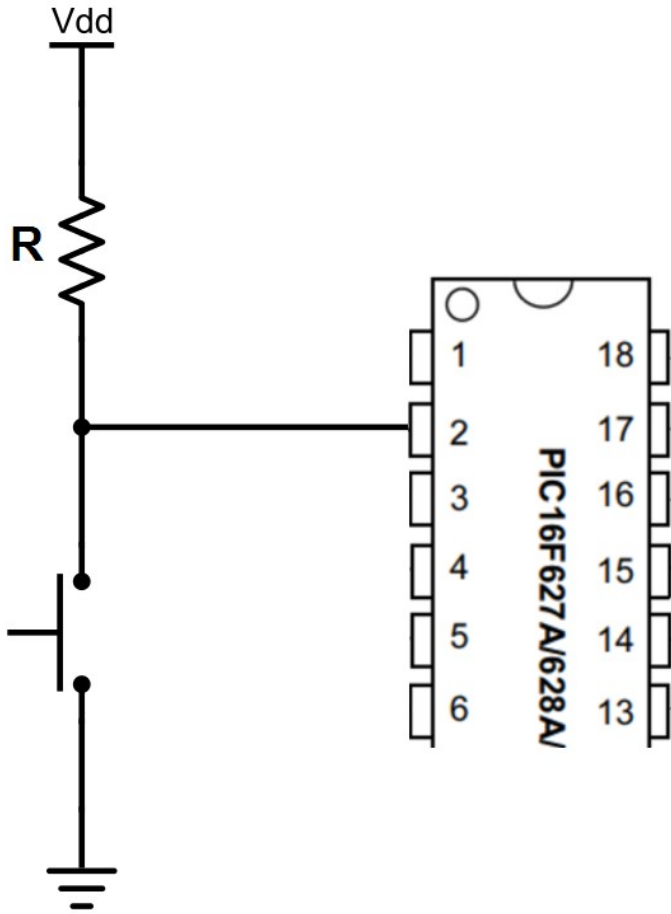
Entrada digitais



Pull-down

O nível alto só é ativado quando o botão é apertado.

Entrada digitais



Pull-up

Um nível alto é colocado na entrada digital que é aterrada quando a chave é apertada

Entrada digitais

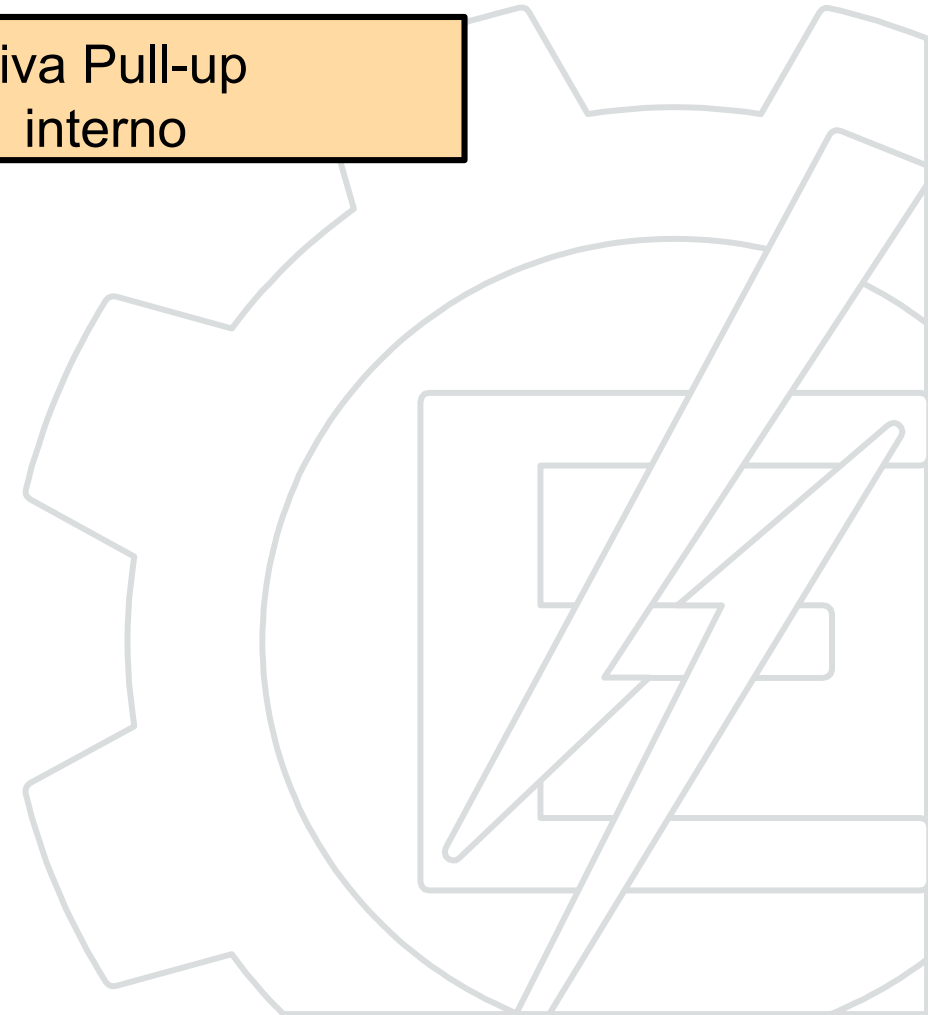
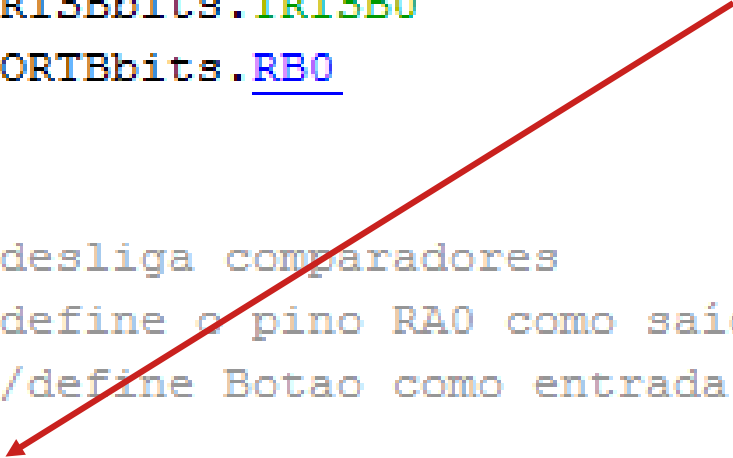
Alguns microcontroladores possuem pull-up internos.

```
#define LED0_TRIS    TRISAbits.TRISA0
#define LED0        PORTAbits.RA0
#define BOTAO_TRIS  TRISBbits.TRISB0
#define BOTAO        PORTBbits.RB0
```

```
- void main(void) {
    CMCON = 0x07; //desliga comparadores
    LED0_TRIS = 0; //define o pino RA0 como saída
    BOTAO_TRIS = 1; //define Botao como entrada

    nRBPU = 0; //Ativa pull-up
```

Ativa Pull-up
interno



Leitura de teclas

- Para fazer a leitura
 - Configurar o terminal como entrada.
 - Testar o bit desejado.

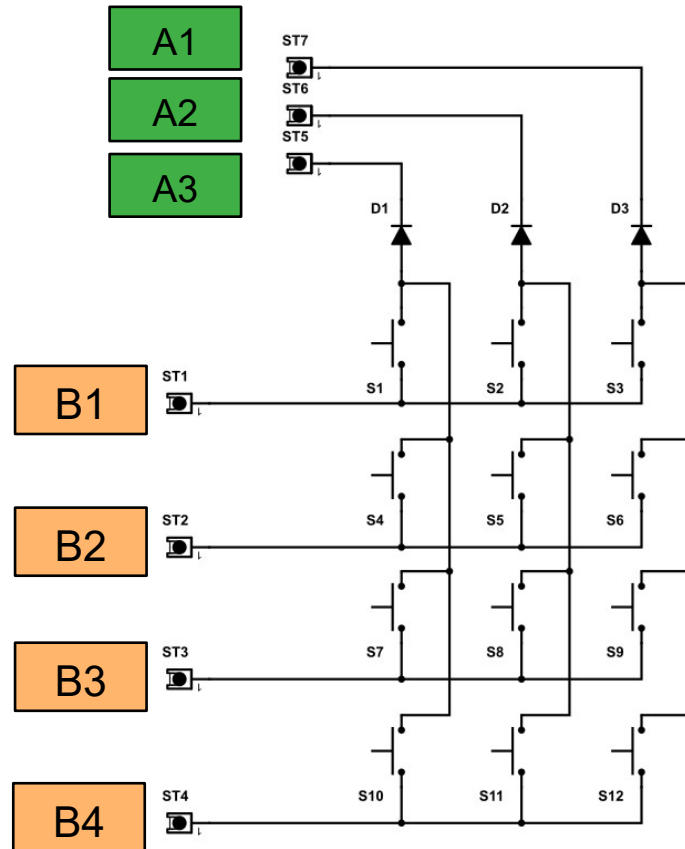
```
void main (void){  
  
    BitSet(TRISD,0); //inicializa como entrada  
    for(;;){  
        if(BitTst(PORTD,0)){  
            //A chave está pressionada  
        }else{  
            //A chave está solta  
        }  
    }  
}
```



O teclado matricial



O teclado matricial



A1, A2, e A3

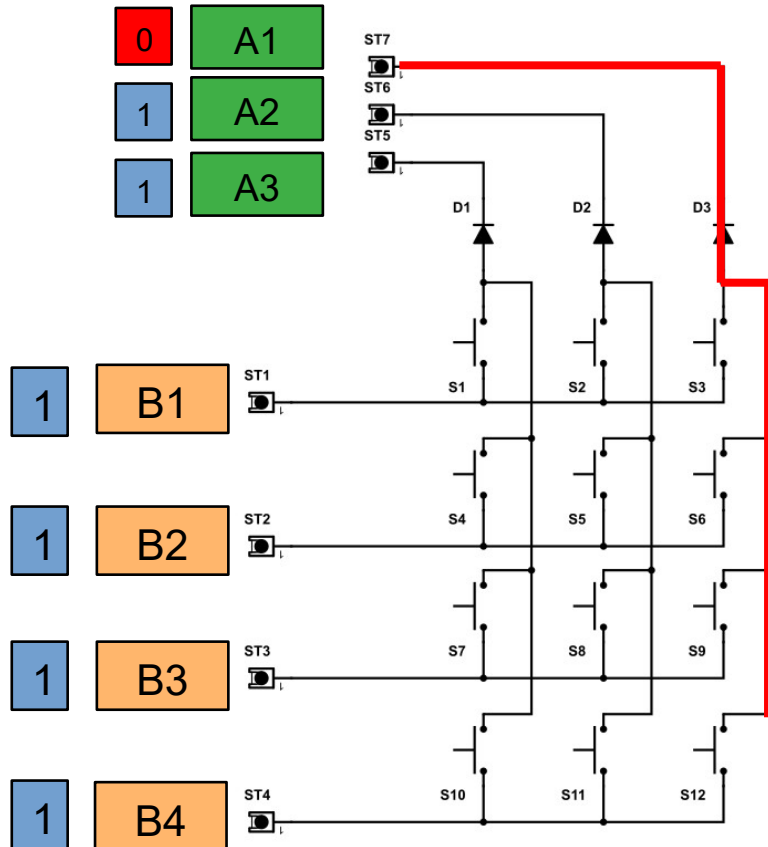
Saídas digitais

B1, B2, B3 e B4

Entradas com pull-up

O teclado matricial permite a utilização de um número de teclas maior que o número de portas.

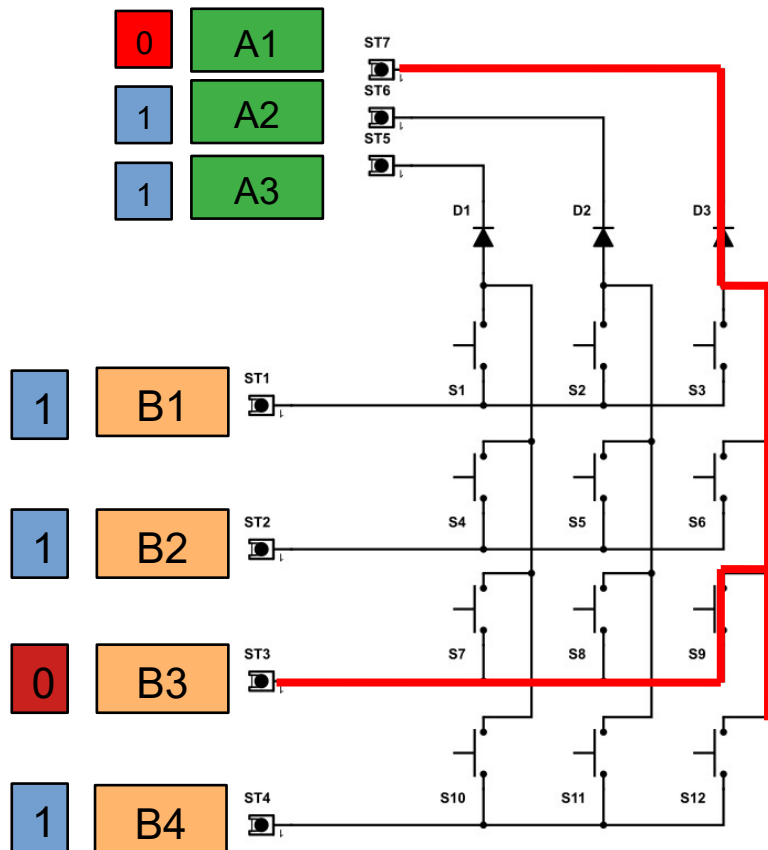
O teclado matricial



Testando linha A1

Se algum botão da linha A1
for apertado a entrada B
correspondente irá para zero

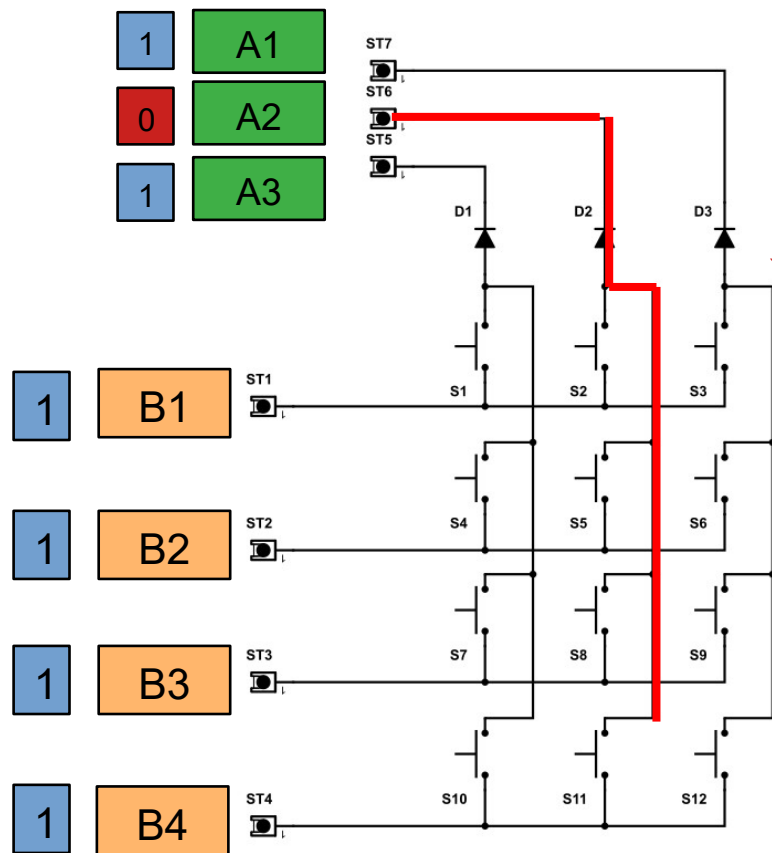
O teclado matricial



Testando linha A1

Neste exemplo, apertando a chave S9 a entrada B3 irá para zero

O teclado matricial

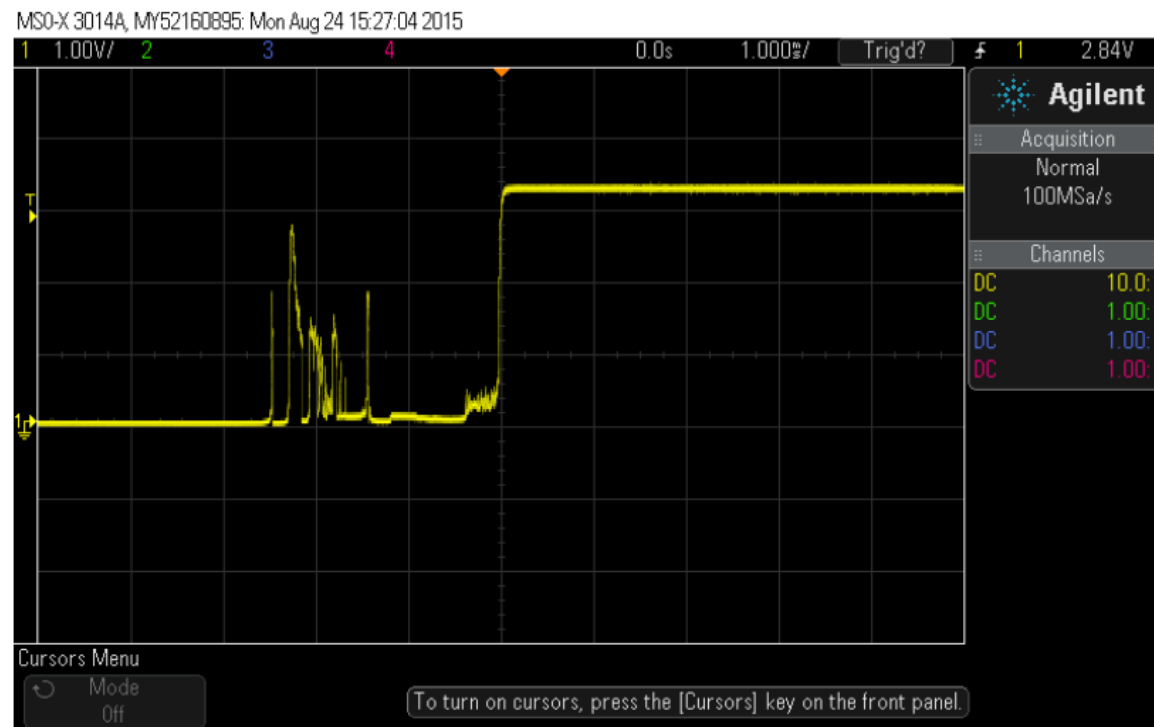
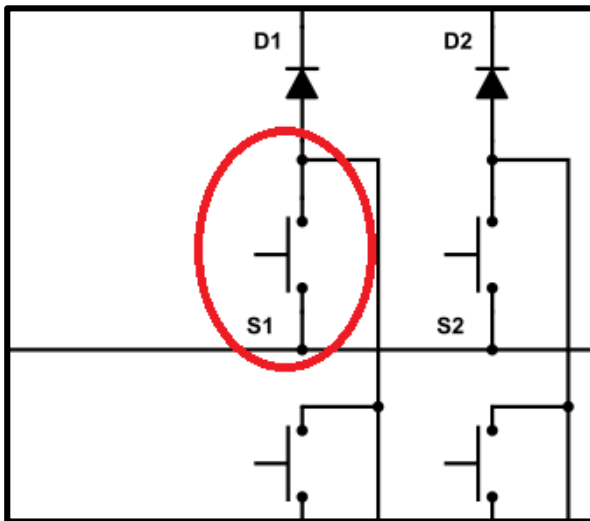


Testando linha A2

Os diodos evitam curto circuito entre A1, A2 e A3 se duas teclas forem apertadas ao mesmo tempo

O bouncing

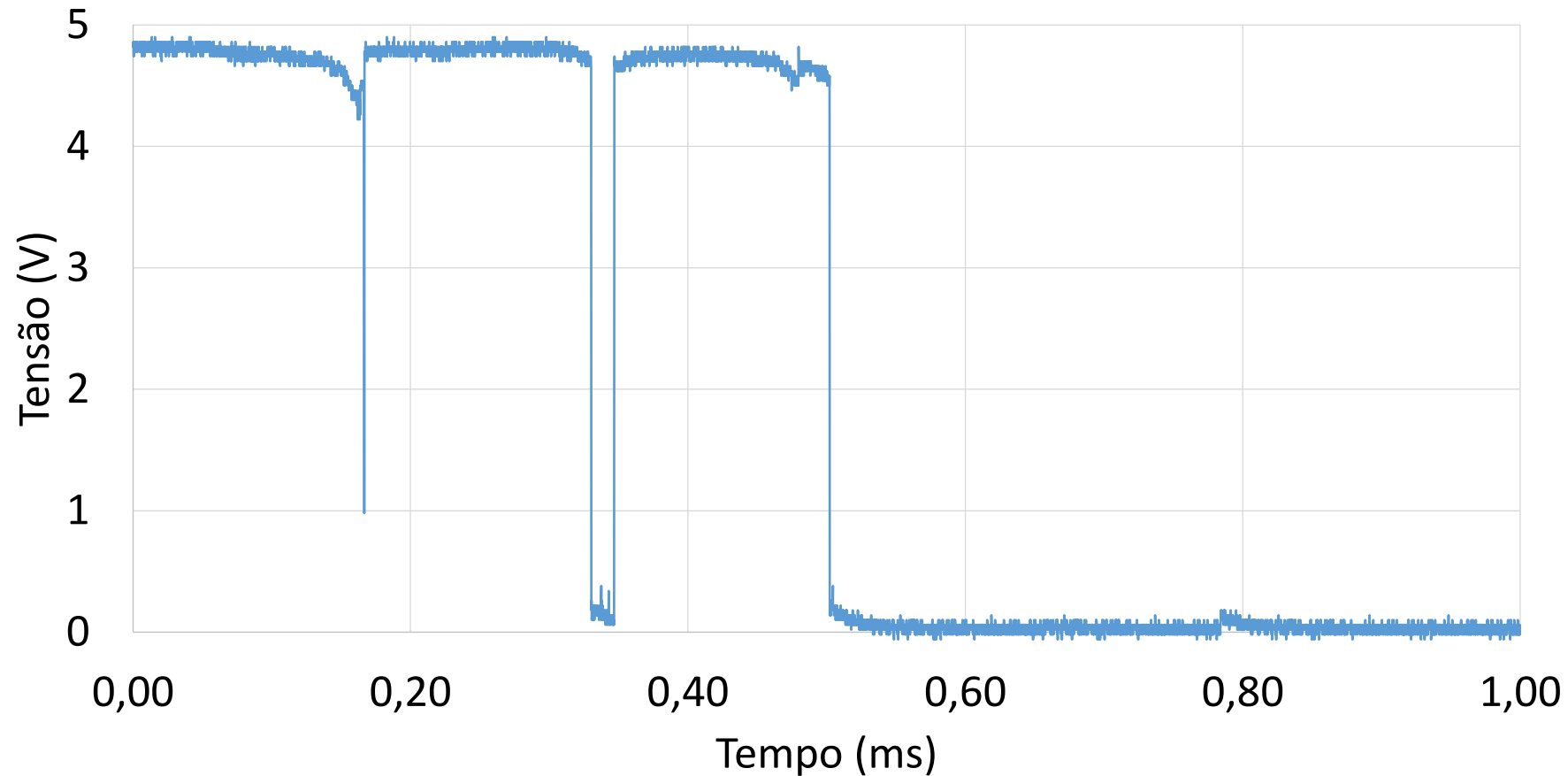
A tecla foi apertada e depois solta



Esse ruído pode aparecer
ao apertar ou ao soltar

Bouncing de uma tecla com pullup

Efeito de bouncing

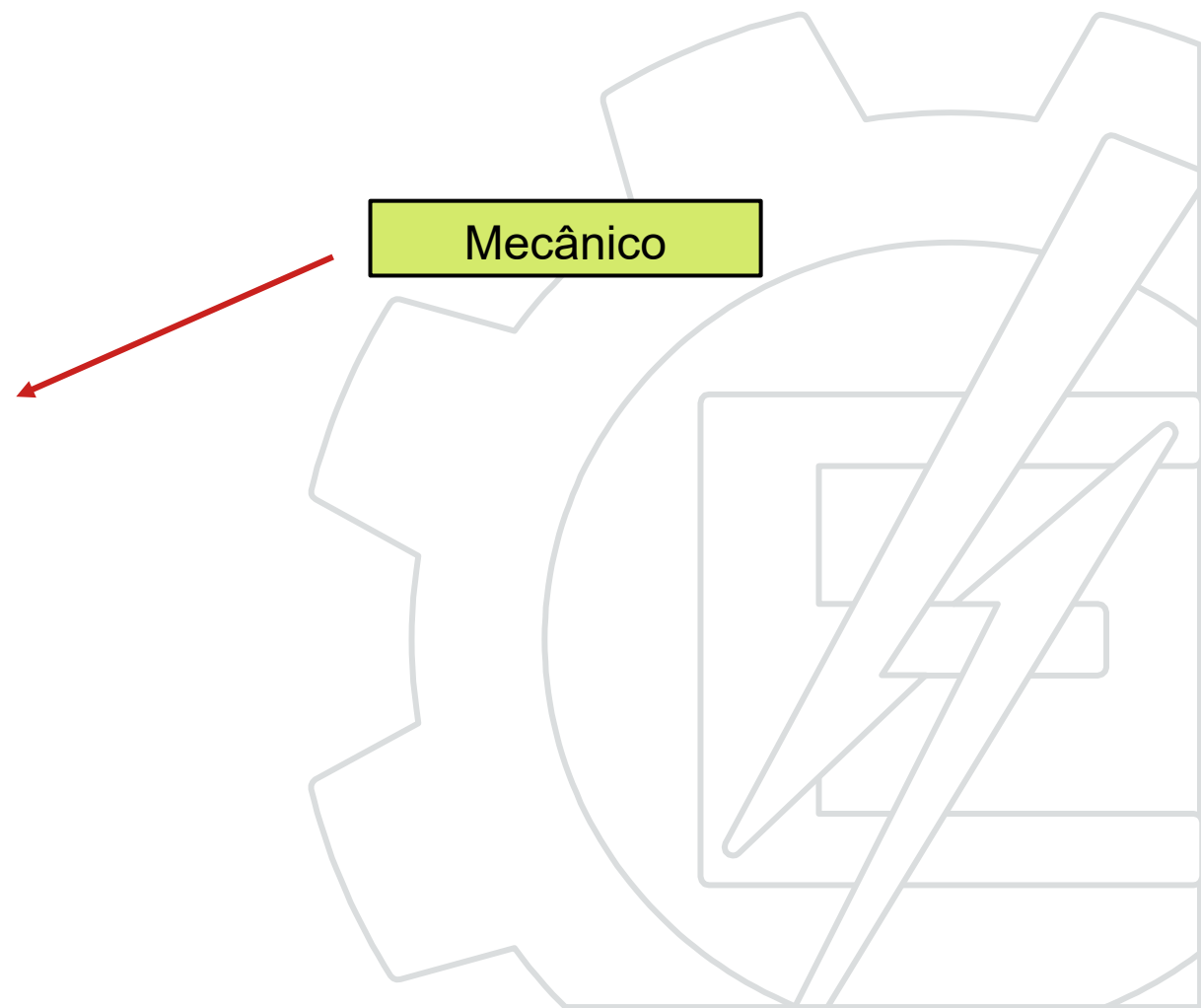


O debounce

Solução por Hardware

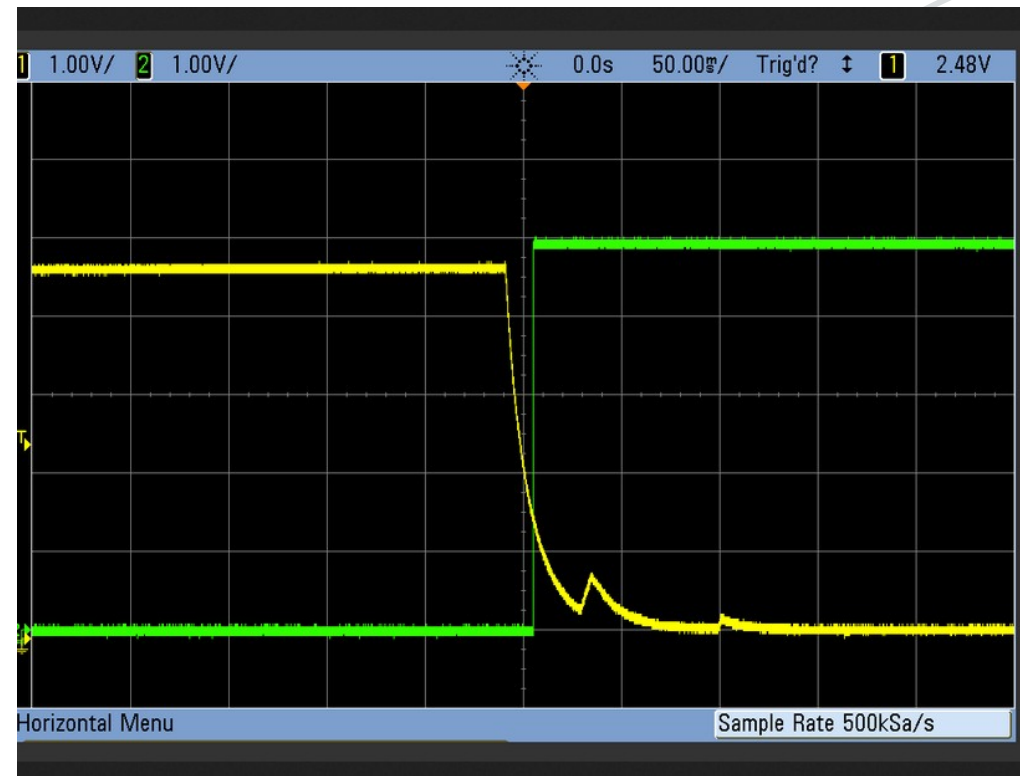
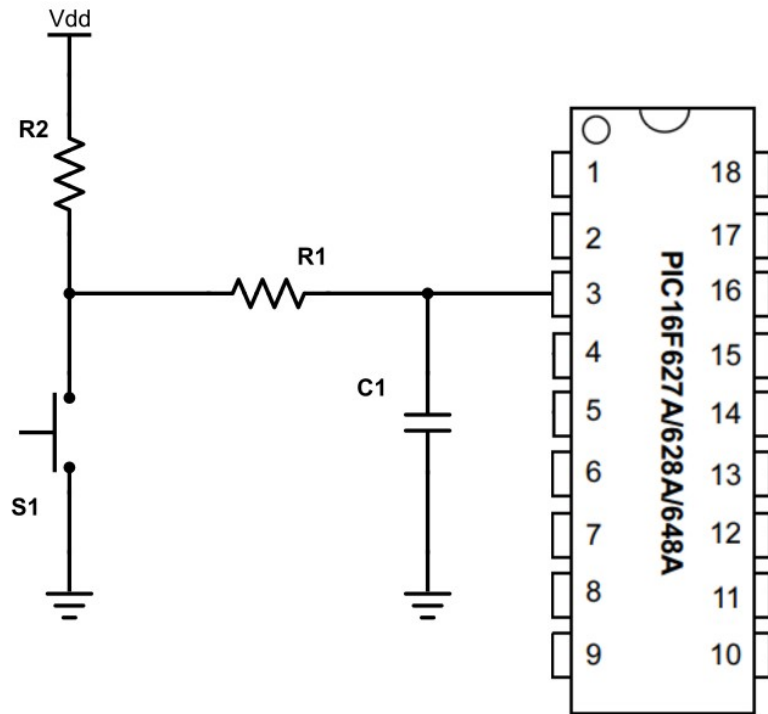


Mecânico



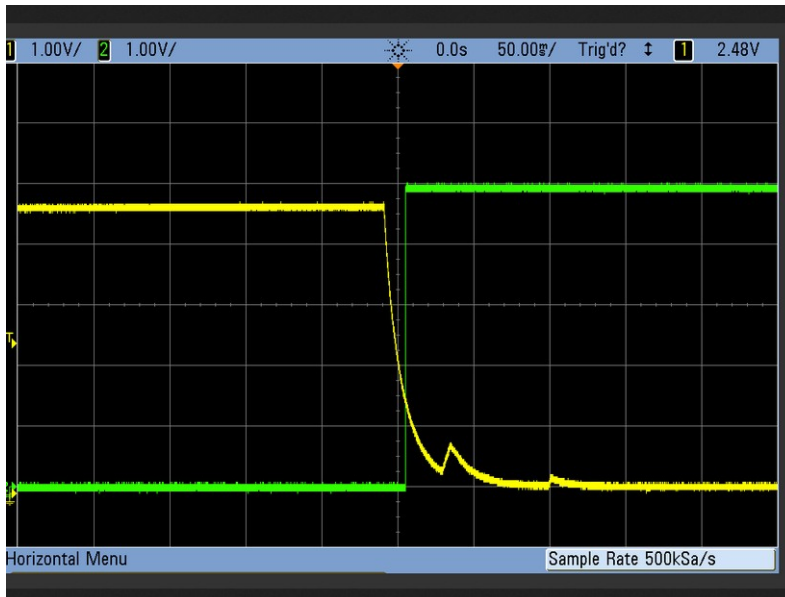
O debounce

Solução por Hardware



O debounce

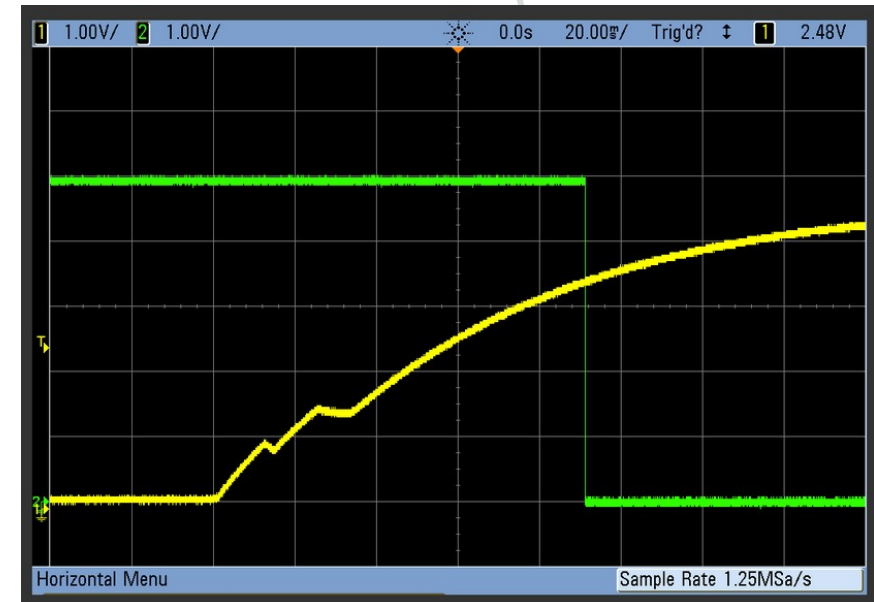
Ajustando a constante de tempo RC



Certo

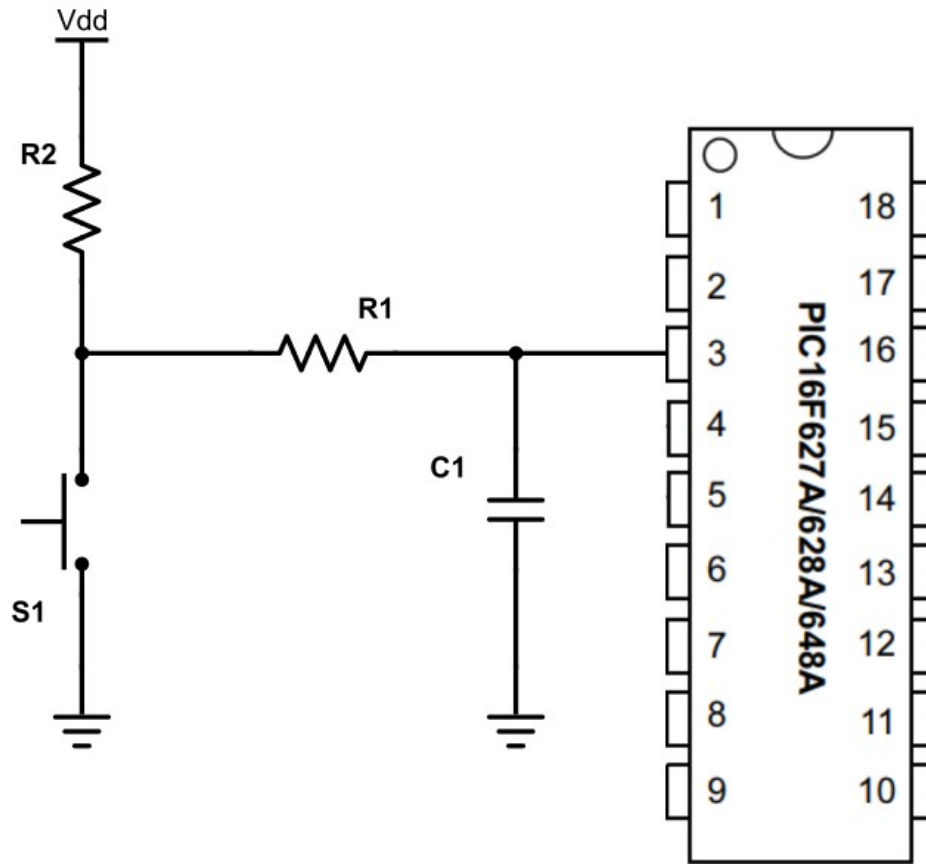


Imagem: wdrfree.com



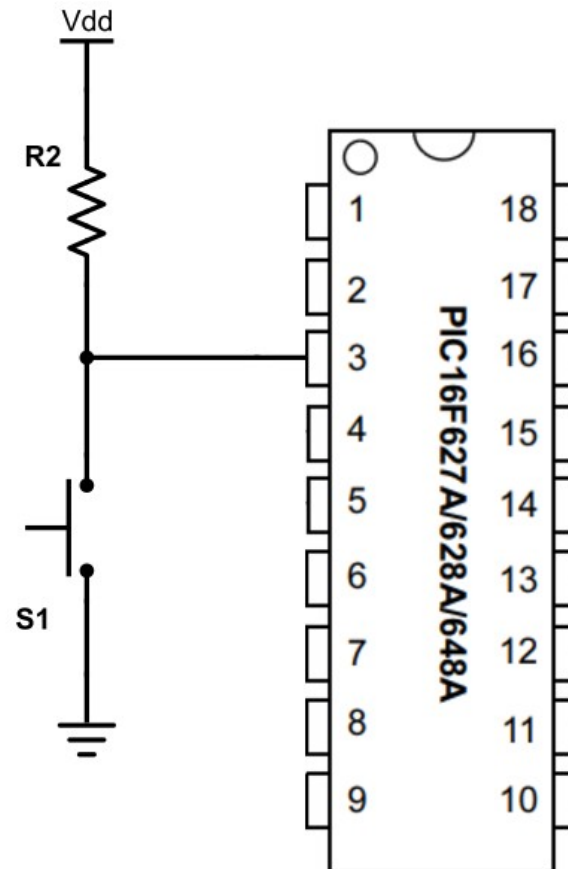
Errado

Debounce por hardware



- Não gasta processamento.
- Protege o circuito contra surtos.
- Gera delay de $R \times C$ na resposta.

O debounce



Solução por Software

- Verificar mudança de estado.
- Não mudar variável.
- Verificar novamente (várias vezes).
- **Se estado confirmado** mudar variável.

Leitura sem debounce

```
void main(void) {  
    CMCON = 0x07; //desliga comparadores  
    LED0_TRIS = 0; //define pino RA0 como saída  
    BOTAO_TRIS = 1; //define botao como entrada  
    LED0 = 0; //Inicializa LED apagado  
  
    nRBPU = 0; //Ativa pull-up  
  
    while(1) {  
        if(BOTAO==1) LED0=1; else LED0=0;  
    }  
    return;  
}
```

Sem debounce
Lê somente uma vez

Leitura com debounce

```
void main(void) {  
  
    unsigned char botao_atual, contador;  
  
    CMCON = 0x07; //desliga comparadores  
    LEDO_TRIS = 0; //define pino RA0 como saída  
    BOTAO_TRIS = 1; //define botao como entrada  
    LEDO = 0; //Inicializa LED apagado  
  
    nRBPU = 0; //Ativa pull-up  
  
    botao_atual=0;  
    contador=0;  
  
    while(1){  
        if(BOTAO!=botao_atual) contador++; else contador=0;  
        if(contador==100)  
        {  
            botao_atual=BOTAO;  
            contador=0;  
        }  
        if(botao_atual) LEDO=1; else LEDO=0;  
    }  
    return;  
}
```

Variáveis de apoio
e inicialização

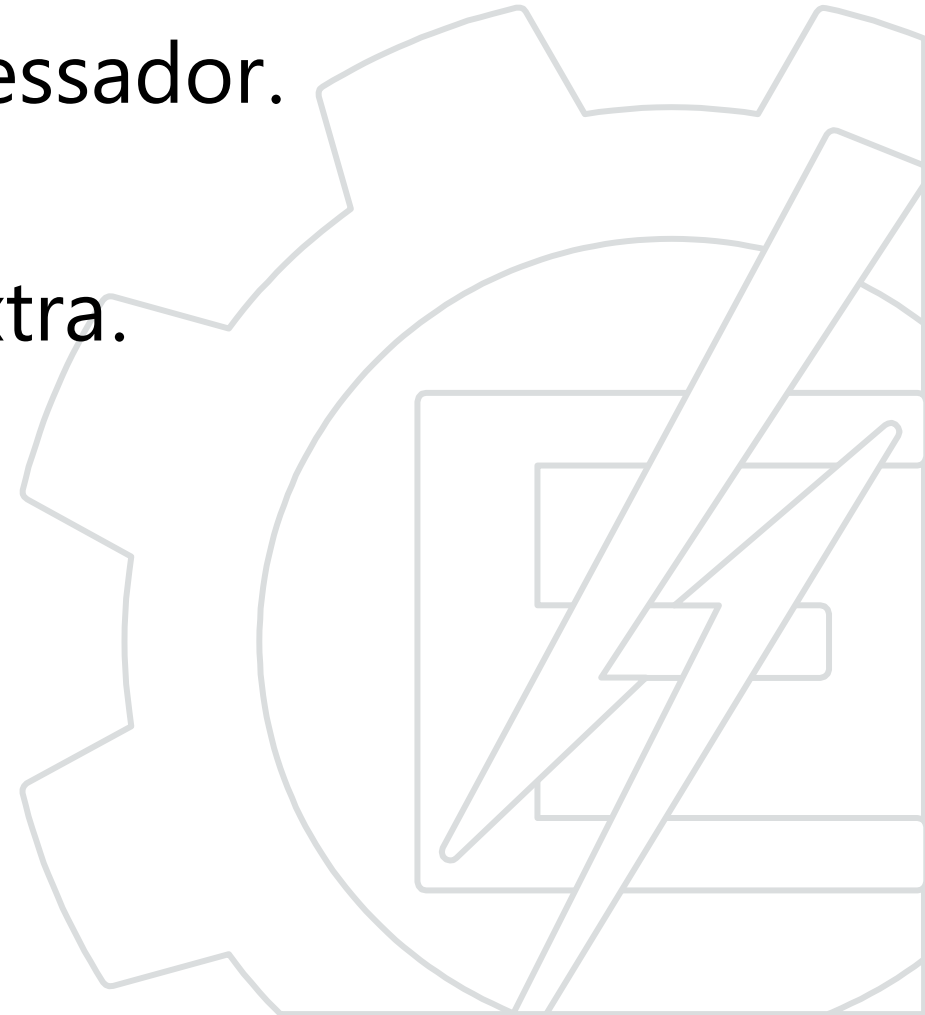
Verifica se mudou

Muda estado atual
e saída

A mudança da variável é realizada
depois de verificar 100 vezes

Debounce por software

- Consome tempo e recurso do processador.
- Não necessita de hardware (\$\$\$) extra.
- Gera delay de acordo com a rotina desenvolvida.

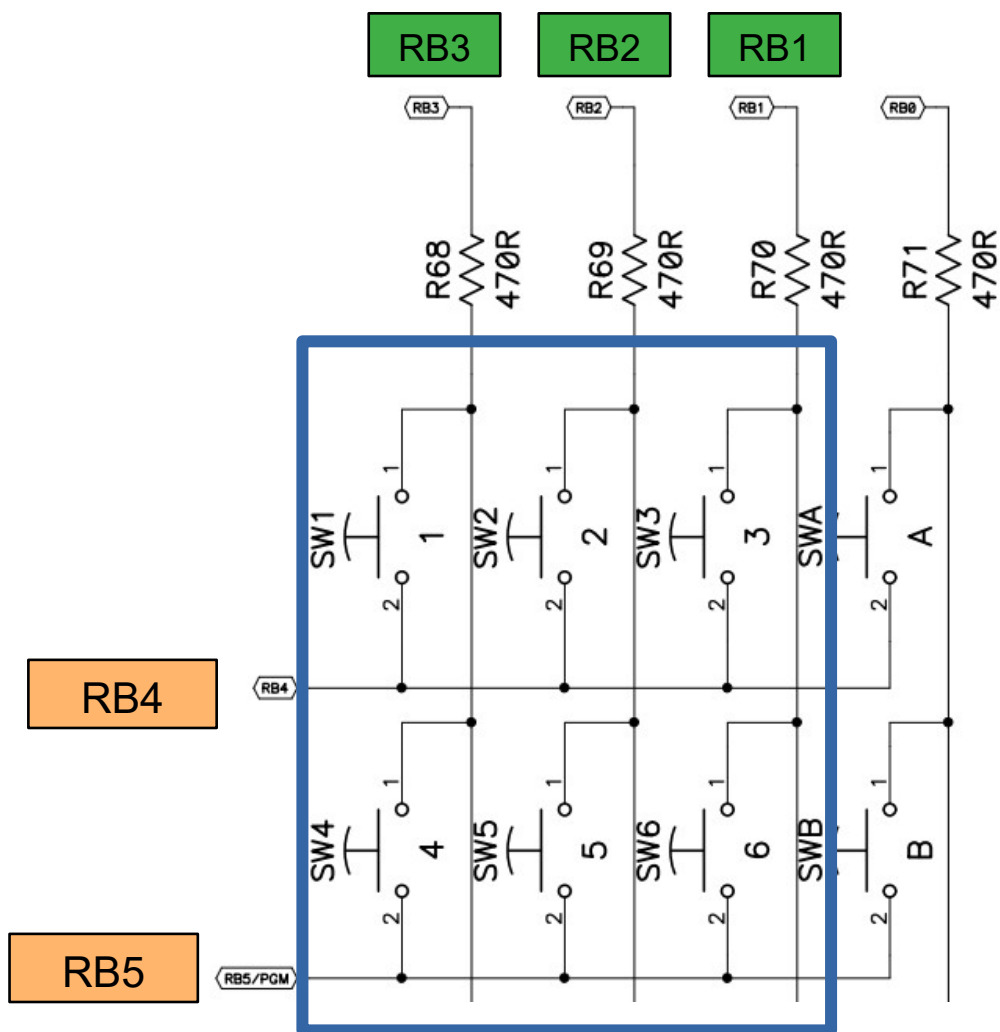


Exemplo de Biblioteca

Leitura do teclado matricial



O teclado matricial



RB3, RB2, e RB1

Saídas digitais

RB4 e RB5

Entradas sem pull-up

```

void main(void){
    unsigned char coluna;
    unsigned char chave[3][2];
    BitClr(TRISB,1); BitClr(TRISB,2); BitClr(TRISB,3);
    BitSet(TRISB,4); BitSet(TRISB,5);
    for(;;){
        for(coluna = 0; coluna < 3; coluna++){
            PORTB = PORTB & 0xF1; //desliga colunas
            BitSet(PORTB,coluna+1); //liga coluna

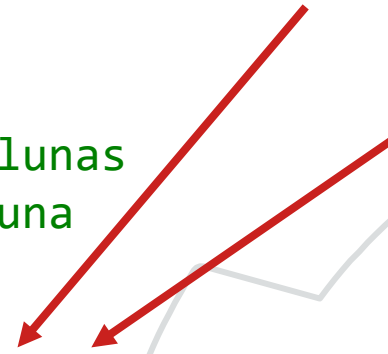
            //testa 1a linha
            if (BitTst(PORTB,4)) chave[coluna][0] = 1;
            else                 chave[coluna][0] = 0;

            //teste da 2a linha
            if (BitTst(PORTB,5)) chave[coluna][1] = 1;
            else                 chave[coluna][1] = 0;
        }
        //Continuação do programa ...
    }
}

```

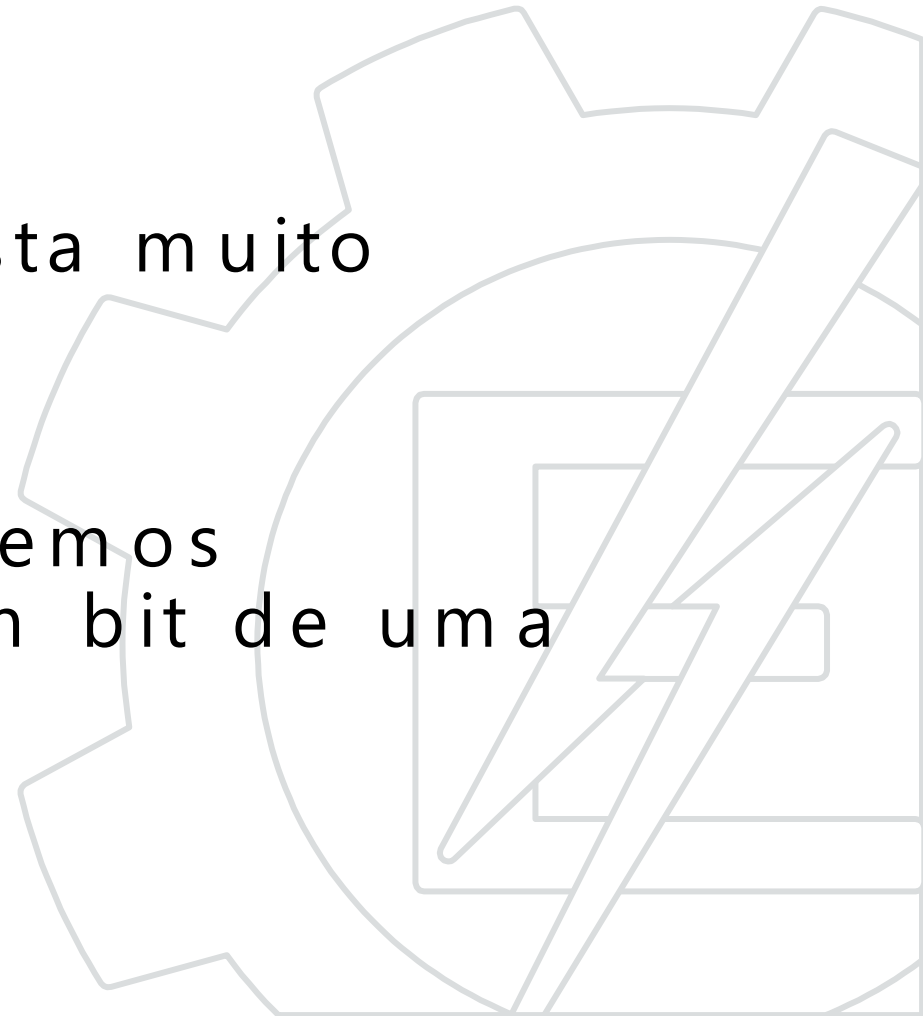
Coluna

Linha



Observação

- O código anterior não apresenta debounce em software para as teclas.
- A operação com uma matriz gasta muito processamento e memória.
- Para minimizar estes gastos podemos representar cada chave como um bit de uma variável.



Exemplo:
Biblioteca keypad



Criação da biblioteca keypad

- Funções criadas:
 - `kpInit()`
 - Configura os terminais de acionamento e de leitura.
 - `kpDebounce()`
 - Realiza a leitura das teclas (varredura).
 - Realiza o debounce dos valores.
 - Armazena internamente as teclas pressionadas em uma variável char.
 - `kpReadkey()`
 - Retorna o valor da variável char.
 - O valor desta variável representa a tecla apertada.

Arquivo keypad.h

```
#ifndef KEYPAD_H
#define KEYPAD_H
    char kpReadkey(void);
    void kpDebounce(void);
    void kpInit(void);
#endif
```



```
#include "keypad.h"
```

Variável com
estado dos botões

```
static unsigned char keys;
```

```
//vetor com o "nome" dos botões
```

```
//U -> up, L -> left, D -> down, R -> right
```

```
//S -> start, s -> select
```

```
static const char charKey[] =
```

```
{'U','L','D','R','S','s'};
```

Nomes dos botões

```
//Iniciando as portas
```

```
void kpInit(void)
{
    BitClr(PORTB,1);
    BitClr(PORTB,2);
    BitClr(PORTB,3);
    BitSet(PORTB,4);
    BitSet(PORTB,5);
}
```



//Lendo o estado dos botões

```
void kpDebounce(void) {  
    int i;  
    static unsigned char tempo;  
    static unsigned char newRead;  
    static unsigned char oldRead;  
    newRead = 0;  
    for(i = 0; i<3; i++){  
        PORTB = PORTB & 0xF1; //RB1,RB2,RB3 = 0  
        BitSet(PORTB,(i+1)); //Liga RB1, RB2...  
        if(BitTst(PORTB,4))BitSet(newRead,i);  
        if(BitTst(PORTB,5))BitSet(newRead,(i+3));  
    }  
    if (oldRead == newRead) tempo--;  
    else {  
        tempo = 4;  
        oldRead = newRead;  
    }  
    if (tempo == 0) keys = newRead;  
}
```

Se igual decrementa

Se zerou atualiza keys

Variável com o estado dos botões

//Retornando o último estado lido

```
char kpReadKey(void){  
    int i;  
    for(i=0;i<6;i++){  
        if BitTst(keys,0)  
            return charKey[i];  
    }  
    //nenhuma tecla pressionada  
    return 0;  
}
```



Exemplo de utilização

```
...  
for(;;){  
    kpDebounce();  
    actualValue = kpReadkey();  
    //houve algum evento?  
    if (actualValue != lastValue){  
        //processa os eventos  
        ...  
    }  
    //armazena a mudança para a próxima rodada  
    lastValue = actualValue;  
}
```

Depois de 4 chamadas
se não mudou atualiza keys

Compara com valor anterior

Até a próxima!

